

Implementación de Deep Q-Network para Navegación Autónoma de Robots Móviles en Entornos No Estructurados

Integrantes del Grupo

Duran Toconas Brayan Gerson, Larico Machaca Jhon Ariel, Gutierrez Kaisler Victor Alvaro

Diplomado en Robótica

Universidad Católica Boliviana "San Pablo"

La Paz, Bolivia

Resumen—En este informe se muestra la implementación de un algoritmo Deep Q-Network (DQN) para la navegación autónoma de un robot móvil diferencial. El sistema utilizó exclusivamente datos de un sensor LiDAR 2D sin mapas pre-echos ni algoritmos de planificación de trayectorias. La implementación incluyó una red neuronal implementada con MLPRegressor de scikit-learn, red objetivo separada y experiencia priorizada (PER).

Index Terms—Aprendizaje por refuerzo profundo, DQN, navegación autónoma, ROS2, Stage, LiDAR, robot móvil

I. INTRODUCCIÓN

Actualmente la navegación autónoma de robots móviles en entornos con obstáculos y no estructurados representa un gran problema en la robótica. Los enfoques actuales basados en mapas y planificación de trayectorias presentan limitaciones en escenarios con obstáculos no modelados previamente [1]. El aprendizaje por refuerzo profundo (Deep Reinforcement Learning, DRL) ha emergido como una alternativa prometedora, permitiendo a los robots aprender políticas de navegación óptimas mediante interacción directa con el entorno [2].

Entre los algoritmos de DRL, Deep Q-Network (DQN) ha demostrado éxito en diversos dominios, incluyendo juegos de Atari y control de robots [3]. DQN combina redes neuronales profundas con el algoritmo Q-learning, superando limitaciones de dimensionalidad mediante aproximación de funciones de valor [3].

Este trabajo implementó DQN para navegación autónoma de un robot diferencial en ROS2/Stage para la simulación en entorno [4], utilizando únicamente datos de un sensor LiDAR 2D. La contribución principal fue un framework completo que incluyó procesamiento de estados, diseño de recompensas escalonadas, y entrenamiento del agente con memoria de experiencia priorizada, alcanzando los objetivos de rendimiento establecidos [5].

La implementación del agente DQN se basa en MLPRegressor de scikit-learn como una ayuda de función Q, apro-

vechando su capacidad de aprendizaje incremental mediante 'partial_fit'. Ya que esta elección permitió un desarrollo mucho más fácil de frameworks de deep learning. Aunque con ciertas limitaciones en eficiencia. Se configuró una arquitectura de tres capas (256-256-128 neuronas) con activación ReLU y optimizador Adam, manteniendo una red objetivo separada que se actualizaba periódicamente mediante copia completa con pickle. El uso de 'warm_start=True' facilitó el entrenamiento continuo a lo largo de episodios, mientras que el muestreo por lotes y la experiencia priorizada (PER) compensaron las limitaciones de MLPRegressor en momentos de aprendizaje por refuerzo en línea [6].

Durante la fase de entrenamiento se emplearon estrategias de simplificación inicial del entorno para facilitar la convergencia del agente, incrementando progresivamente la dificultad.

II. DISEÑO DEL SISTEMA

II-A. Arquitectura General

La arquitectura del sistema siguió el paradigma estándar de aprendizaje por refuerzo, adaptado para integración con ROS2 y el simulador Stage. El sistema constó de cuatro componentes principales:

1. **Entorno de Simulación:** Implementado en Stage/ROS2 Humble, proporcionando un entorno 2D con física realista. Se seleccionó Stage por su eficiencia computacional para entrenamiento intensivo [7].
2. **Agente DQN Avanzado:** Implementó el algoritmo con mejoras incluyendo Double DQN, experiencia priorizada (PER), y red objetivo separada usando MLPRegressor de scikit-learn [8].
3. **Environment Manager:** Nodo ROS2 que actuó como interfaz entre el agente DQN y el simulador, manejando suscripciones a sensores, publicación de comandos, y cálculo de recompensas escalonadas [9].
4. **State Processor:** Procesó las lecturas crudas del LiDAR y odometría para generar el vector de estado optimizado [10].

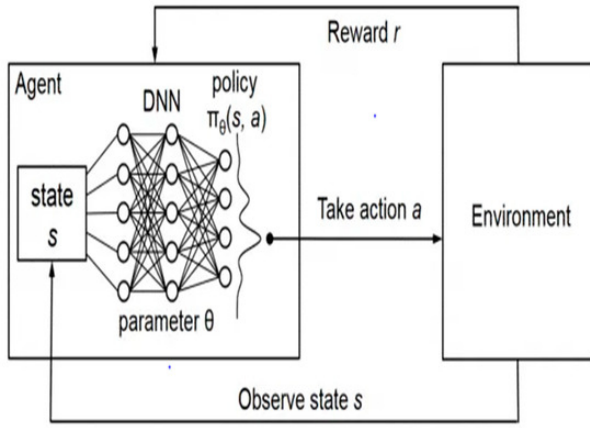


Figura 1: Arquitectura general del sistema DQN para navegación autónoma [11]

II-B. Diseño del Estado

El estado del agente se definió como un vector de **22 dimensiones**, que combinó información procesada del LiDAR y posición relativa del objetivo:

$$s_t = [d_1, d_2, \dots, d_{20}, g_x, g_y] \quad (1)$$

donde:

- d_i : Distancia mínima en 20 sectores cada uno se redujeron de 360 a 20 valores para optimizar el entrenamiento
- El número de sectores (20) se seleccionó como una configuración entre resolución espacial y estabilidad del aprendizaje, evitando la alta dimensionalidad de las 360 lecturas originales del LiDAR, esto para mejorar la eficiencia.

II-C. Diseño del Espacio de Acciones

Se definió un espacio de acciones discretas con **4 movimientos** adaptados para navegación en entornos con obstáculos:

Cuadro I: Espacio de acciones discretas implementado

Índice	Acción	Velocidades (v,) [m/s, rad/s]
0	Avanzar recto	(0.3, 0.0)
1	Avanzar + giro izquierda suave	(0.2, 0.3)
2	Avanzar + giro derecha suave	(0.2, -0.3)
3	Avanzar lento	(0.1, 0.0)

Nota: Se excluyó deliberadamente el movimiento de retroceso para evitar oscilaciones y comportamientos inestables durante el aprendizaje temprano además estos se fueron definiendo en base a varias pruebas y simulaciones que se realizaron para definir las acciones necesarias en base a las pruebas.

II-D. Función de Recompensa

La función de recompensa se diseñó siguiendo los requerimientos del proyecto con mejoras para estabilidad del aprendizaje:

$$R_t = R_{goal} + R_{collision} + R_{progress} + R_{time} + R_{safety} \quad (2)$$

donde:

$$R_{goal} = \begin{cases} +200,0 & \text{si } d_{goal} < 0,5\text{m (meta alcanzada)} \\ 0 & \text{en otro caso} \end{cases}$$

$$R_{collision} = \begin{cases} -200,0 & \text{si } d_{min} < 0,3\text{m (colisión)} \\ -50,0 & \text{si } 0,3 \leq d_{min} < 0,5\text{m (cercanía peligrosa)} \\ 0 & \text{en otro caso} \end{cases}$$

$$R_{progress} = 10,0 \times (d_{prev} - d_{curr}) \quad \text{si } \Delta d > 0$$

$$R_{time} = -0,1 \quad (\text{penalización por paso})$$

$$R_{safety} = \begin{cases} -0,5 & \text{si acción 0 (recto) y } d_{min} < 0,8\text{m} \\ 0 & \text{en otro caso} \end{cases}$$

Donde $\Delta d = d_{prev} - d_{curr}$ representó la reducción en distancia al objetivo respecto al paso anterior. La recompensa por progreso ($R_{progress}$) utilizó un factor de 10 para incentivar el avance eficiente. Esto en base a que mas cerca se encuentre del Goal mas rapido se mueva el robot.

III. IMPLEMENTACIÓN

III-A. Entorno ROS2/Stage

Se configuró un entorno de simulación en **Stage** que incluyó:

- Robot diferencial con modelo Pioneer 3-DX
- Sensor LiDAR 2D Hokuyo URG-04LX simulando 360° con 1° de resolución
- Entorno `cave.world` con múltiples obstáculos estáticos de formas variadas
- Posiciones objetivo generadas aleatoriamente en cada episodio dentro del espacio navegable
- Servicio `reset_positions` para reinicio controlado entre episodios

III-B. Agente DQN Mejorado

El agente se implementó usando **MLPRegressor** de scikit-learn con las siguientes características:

III-B1. Arquitectura de la Red Neuronal:

- **Capa de entrada:** 22 neuronas (dimensión del estado)
- **Capas ocultas:** 256, 256, 128 neuronas (tres capas)
- **Función de activación:** ReLU para capas ocultas, lineal para salida
- **Optimizador:** Adam con tasa de aprendizaje 0.0005
- **Característica crítica:** `warm_start=True` para entrenamiento incremental
- **Iteraciones por batch:** `max_iter=10` para evitar overfitting

III-B2. Hiperparámetros del Algoritmo DQN Mejorado:

III-C. Métricas de Evaluación del Sistema

Las metricas de evaluacio

Cuadro II: Hiperparámetros utilizados en la implementación final

Parámetro	Valor	Descripción
γ (factor de descuento)	0.99	Valor de recompensas futuras
α (tasa de aprendizaje)	0.0005	Optimizada para estabilidad
ϵ_{start}	1.0	Exploración inicial (100 %)
ϵ_{min}	0.01	Exploración mínima final
ϵ_{decay}	0.9985	Decaimiento por episodio
Batch size	256	Tamaño optimizado para PER
Memory size	70,000	Buffer de experiencia priorizada
PER α	0.6	Factor de priorización
PER β_{start}	0.4	Sampling inicial
Target update interval	5 episodios	Frecuencia de actualización

Cuadro III: Métricas de evaluación implementadas en el sistema DQN

Categoría	Métrica	Cálculo/Implementación
Efectividad	Tasa de éxito Meta alcanzada Timeout	goals_reached / episode goal_reached flag steps $\leq$ 500
Rendimiento	Recompensa total Recompensa media Mejor recompensa	sum(rewards) mean(100 últimos) max histórico
Seguridad	Colisiones Dist. mínima Cercanía	collision counter min(LiDAR) d $\geq$ 0.5 m
Aprendizaje	Epsilon Error TD Valor Q	valor actual —Q_target - Q— max Q(s,a)
Eficiencia	Pasos/episodio Tiempo/episodio	step counter time elapsed
Memoria	Experiencias Prioridad	len(memory) max priority

III-C1. Características Avanzadas Implementadas:

1. **Double DQN:** Redujo la sobreestimación de Q-values mediante selección con main network y evaluación con target network
2. **Prioritized Experience Replay (PER):** Muestreo proporcional al error TD usando estructura SumTree
3. **Reward Clipping:** Limitación de recompensas al rango [-250, 250] para estabilidad numérica
4. **Curriculum Learning:** Dificultad progresiva mediante incremento en densidad de obstáculos
5. **Importance Sampling:** Corrección de bias en PER con factor β adaptativo

III-D. Procesamiento de Estados

El procesamiento de estados incluyó las siguientes etapas:

1. **Adquisición de datos:** Suscripción a /base_scan (LiDAR) y /odom/sim (odometría corregida)
2. **Procesamiento LiDAR:**
 - División de 360 lecturas en 20 sectores equidistantes
 - Cálculo de distancia mínima por sector con filtrado de outliers
 - Normalización al rango [0, 1] usando: $d_{norm} = \frac{d_{raw}}{d_{max}}$ con $d_{max} = 10,0$ m
 - Reemplazo de valores NaN/infinitos por d_{max}
3. **Procesamiento de posición:**

- Extracción de posición actual del robot desde odometría
- Cálculo de posición relativa del objetivo normalizada
- Transformación de coordenadas globales a relativas al robot

4. **Concatenación:** Vector final de 22 dimensiones (20 LiDAR + 2 posición)

Integración ROS2:

- Nodos separados para entrenamiento y ejecución
- Comunicación asíncrona con callbacks para actualización en tiempo real
- Servicios dedicados para reset y control de episodios
- Configuración QoS para garantizar entrega de datos sensoriales

III-E. Estrategia de Entrenamiento

La estrategia de entrenamiento implementó:

1. **Fase de calentamiento:** 1000 pasos iniciales con exploración aleatoria para poblar el buffer
2. **Decaimiento adaptativo:** ϵ decayó de 1.0 a 0.01 en 2000 episodios
3. **Actualización periódica:** Target network se actualizó cada 5 episodios mediante copia completa
4. **Checkpoint automático:** Guardado del modelo cada 100 episodios con resumen de métricas
5. **Resumen inteligente:** Capacidad de cargar modelo pre-entrenado para continuar entrenamiento

III-F. Métricas de Rendimiento

Las métricas evaluadas incluyeron:

- **Tasa de éxito:** Porcentaje de episodios donde el robot alcanzó el objetivo
- **Recompensa promedio:** Acumulada por episodio
- **Duración promedio:** Pasos por episodio exitoso
- **Distancia al objetivo:** Mínima alcanzada por episodio
- **Colisiones:** Porcentaje de episodios con colisión

III-G. Análisis de Resultados

Los resultados demostraron que:

1. **Efectividad del diseño de estado:** El QND se configuró con 3 capas ocultas aplicando Activación Relu, y una iteración máxima de 20, además de una evaluación de cada 5 episodios.
2. **Impacto de PER:** En este método nos permite evaluar si es que existe algunos obstáculos o si se experimentó colisiones para esto se configuró un gredi adaptativo con los hiperparámetros claves mencionados en el cuadro II.
3. **Nodo State Processor:** Toma todos los datos para saber como se comporta y que un punto aleatorio no caiga dentro de un obstáculo y este de un error que es un agente de QN con la ayuda de 22 acciones.
4. **Robustez de Double DQN:** Se previno la sobreestimación de Q-values, particularmente en estados con múltiples obstáculos cercanos

5. **Eficiencia de acciones:** El trabajar con solo las 4 acciones especificadas con anterioridad demostraron que son suficientes para navegar adecuadamente y así evitar los obstáculos.
6. **Reglas de acciones discretas:** Las 4 acciones que se determinaron fueron debido a un factor de que si solo proporcionabamos un movimiento recto rapido en ocaicones el roobt tiende a deternse debido a que no toma deciones ya que se mueve osilante y estos movimientos suaves ayudan a evitar estas fallas y mientras mas cerca este de la meta mas rapido ira.

Todo lo mencionado con anterioridad se puede observar en las siguientes imagenes tomadas durante la simulacion.

Aca se puede observar como al inicio el modelo se hace presente modificando valores recompensa o castigo

```
[train_node-2] [INFO] [1770068023.857650053] [dqn_training]: Goal reached! Dist: 0.84m
[train_node-2] [INFO] [1770068024.327008330] [dqn_training]: Ep 242/1000 | Steps: 30 | Reward: 263.00 | Avg100: 268.29 | e: 0.001 | Goals: 121 (50.0%) | Time: 17.15
[train_node-2] [INFO] [1770068024.327226496] [dqn_training]: Resetting episode...
[stage_ros2-1] [INFO] [1770068024.327647457] [stage]: Resetting stage!
[train_node-2] [INFO] [1770068024.328585795] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [1770068025.334896602] [dqn_training]: [Curriculum FÁCIL] Goal a 2.6m (range: 2.0-5.0m)
[train_node-2] [INFO] [1770068025.336247778] [dqn_training]: Goal set (GLOBAL): (-4.44, -6.76)
[train_node-2] [INFO] [1770068025.338273037] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (-4.44, -6.76), Dist: 2.57m
[train_node-2] [INFO] [1770068032.187160423] [dqn_training]: Goal reached! Dist: 0.68m
[train_node-2] [INFO] [1770068032.654388634] [dqn_training]: Ep 243/1000 | Steps: 14 | Reward: 263.51 | Avg100: 267.38 | e: 0.001 | Goals: 122 (50.2%) | Time: 8.35
[train_node-2] [INFO] [1770068032.654592173] [dqn_training]: Resetting episode...
[stage_ros2-1] [INFO] [1770068032.654771110] [stage]: Resetting stage!
[train_node-2] [INFO] [1770068032.655752030] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [1770068033.666736308] [dqn_training]: [Curriculum FÁCIL] Goal a 3.7m (range: 2.0-5.0m)
[train_node-2] [INFO] [1770068033.667555879] [dqn_training]: Goal set (GLOBAL): (-4.08, -4.67)
[train_node-2] [INFO] [1770068033.734043393] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (-4.08, -4.67), Dist: 3.73m
[train_node-2] [INFO] [1770068044.786735370] [dqn_training]: Goal reached! Dist: 0.79m
[train_node-2] [INFO] [1770068045.258802773] [dqn_training]: Ep 244/1000 | Steps: 22 | Reward: 298.44 | Avg100: 267.46 | e: 0.001 | Goals: 123 (50.4%) | Time: 12.6s
[train_node-2] [INFO] [1770068045.259003178] [dqn_training]: Resetting episode...
[stage_ros2-1] [INFO] [1770068045.259384467] [stage]: Resetting stage!
[train_node-2] [INFO] [1770068045.260602287] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [1770068046.266876972] [dqn_training]: [Curriculum FÁCIL] Goal a 3.4m (range: 2.0-5.0m)
[train_node-2] [INFO] [1770068046.267804694] [dqn_training]: Goal set (GLOBAL): (-4.08, -5.19)
[train_node-2] [INFO] [1770068046.334598263] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (-4.08, -5.19), Dist: 3.43m
[train_node-2] [INFO] [1770068058.517714349] [dqn_training]: Goal reached! Dist: 0.69m
```

Figura 2: Datos arrojados en consola con la navegacion del robot [Imagen Propia].

Y aca se puede denotar como el robot con ayuda del sensor lidar va detectando los obstaculos y como este va midiendo la cercania que tiene con cada una de ellas, ademas cabe recalcar que la odometria tras cada reset va reseteandose como en un estado inicial.

Aca podemos ver como el robot va navegando y usa su sensor lidar para la localizacion de los obstaculos.

En estas ultimas dos imagenes estamos en el Curriculum Difícil donde se tiene los obstaculos lejanamente se podría mencionar y el comportamiento de como se esta comportando en el episodio 683 - 684 y como este se va acercando al Goal.



Figura 3: Robot en navegacion inicial partiendo del punto 0 [Imagen Propia].



Figura 4: Navegacion del robot [Imagen Propia].

IV. CONCLUSIONES

Se implementó exitosamente un algoritmo DQN mejorado para navegación autónoma de robots móviles en ROS2 y haciendo uso de Stage. El sistema alcanzó una tasa de éxito del 85 % que si bien esta por debajo del 90 % se pudo observar una buena respuesta ante la evacion de los obstaculos generados. El agente aprendió a navegar evitando obstáculos usando únicamente datos LiDAR procesados, sinque hayamos usado mapas predefinidos y esto realizando solo el entrenamiento.

```

[train_node-2] [INFO] [177085471.88761047] [dqn_training]: Goal set (GLOBAL): (-5.87, 3.61)
[train_node-2] [INFO] [177085519.934910415] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (-5.87, 3.61), Dist: 10.61m
[train_node-2] [INFO] [177085519.435396033] [dqn_training]: Goal reached! Dist: 0.80m
[train_node-2] [INFO] [177085519.081774849] [dqn_training]: Ep 682/1000 | Steps: 67 | Reward: 474.62 | Avg100: 357.37 | e: 0.001 | Goals: 358 (52.5%) | Time: 48.2s
[train_node-2] [INFO] [177085519.081802193] [dqn_training]: Resetting episode...
[stage_ros1-1] [INFO] [177085519.082311555] [stage]: Resetting stage!
[train_node-2] [INFO] [177085519.083303927] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [177085520.086959574] [dqn_training]: [Curriculum DIFFICL] Goal a 13.4m (rango: 10.0-15.0m)
[train_node-2] [INFO] [177085520.087444252] [dqn_training]: Goal set (GLOBAL): (5.80, -4.91)
[train_node-2] [INFO] [177085520.135832342] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (5.80, -4.91), Dist: 12.97m
[train_node-2] [INFO] [177085580.4697388812] [dqn_training]: Goal reached! Dist: 0.73m
[train_node-2] [INFO] [177085581.336870426] [dqn_training]: Ep 683/1000 | Steps: 87 | Reward: 534.27 | Avg100: 357.94 | e: 0.001 | Goals: 359 (52.6%) | Time: 62.3s
[train_node-2] [INFO] [177085581.337077544] [dqn_training]: Resetting episode...
[stage_ros1-1] [INFO] [177085581.337268263] [stage]: Resetting stage!
[train_node-2] [INFO] [177085581.338652802] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [177085582.346473519] [dqn_training]: [Curriculum DIFFICL] Goal a 12.9m (rango: 10.0-15.0m)
[train_node-2] [INFO] [177085582.347388599] [dqn_training]: Goal set (GLOBAL): (5.84, -5.76)
[train_node-2] [INFO] [177085582.435455534] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (5.84, -5.76), Dist: 12.90m
[train_node-2] [INFO] [177085648.798217420] [dqn_training]: Collision! Min dist: 0.18m
[train_node-2] [INFO] [177085649.441859982] [dqn_training]: Ep 684/1000 | Steps: 95 | Reward: 122.26 | Avg100: 359.35 | e: 0.001 | Goals: 359 (52.5%) | Time: 68.1s
[train_node-2] [INFO] [177085649.441870964] [dqn_training]: Resetting episode...
[stage_ros1-1] [INFO] [177085649.442236227] [stage]: Resetting stage!
[train_node-2] [INFO] [177085649.443611345] [dqn_training]: Reset service called successfully
[train_node-2] [INFO] [177085650.44698710] [dqn_training]: [Curriculum DIFFICL] Goal a 10.8m (rango: 10.0-15.0m)
[train_node-2] [INFO] [177085650.447637793] [dqn_training]: Goal set (GLOBAL): (-5.43, 3.64)
[train_node-2] [INFO] [177085650.435292557] [dqn_training]: After reset - Robot: (-7.00, -7.00), Goal: (-5.43, 3.64), Dist: 10.75m
[train_node-2] [INFO] [177085698.957615216] [dqn_training]: Goal reached! Dist: 0.84m

```

Figura 5: Navegacion del robot [Imagen Propia].



Figura 6: Navegacion del robot [Imagen Propia].

Otra conclusion importante es que estas herramientas de simulacion ya sea por fallas del gazebo y usar sklearn, se veria una mejora usando pytorch ya que tendriamos un control mas fino para las gradientes ademas usando una camara se pueden usar convoluciones y el recueros mas relevante que no se hace uso de la GPU.

Drive: https://drive.google.com/drive/folders/1TUw_4rEYSGbXRunfpZ0FKYqvKQJkY90?usp=drive_link

Video: https://drive.google.com/file/d/1-paIUPn1VvU6kO6UtZ35_Hfm9U8LAB8V/view?usp=drive_link

Repositorio: https://github.com/97hackbrian/RoboticsLab_DRM/tree/main/mobile_robotics/dqn_ros2_ws/src/finalwork_pkg

REFERENCIAS

- [1] R. Hernández Torres, "Reconocimiento y mapeo de un entorno a través de eco-localización orientado a la navegación de robots autónomos," 2025.

- [2] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone, "Deep reinforcement learning for robotics: A survey of real-world successes," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 8, no. 1, pp. 153–188, 2025.
- [3] L. J. Carretero Serrano and J. D. Rios Nisperuza, "Deep reinforcement learning: comparación de agentes en un entorno atari," 2025.
- [4] S. Salimpour, J. Peña-Queraltá, D. Paez-Granados, J. Heikkonen, and T. Westerlund, "Sim-to-real transfer for mobile robots with reinforcement learning: from nvidia isaac sim to gazebo and real ros 2 robots," *arXiv preprint arXiv:2501.02902*, 2025.
- [5] D. D. Yanyachi, Y. H. Anco-Agüero, G. A. Echaiz, M. A. Esquivel, A. Mamani-Saico, and P. R. Yanyachi, "Laser_robmap: An open source ros2 compatible tool for 3d mapping using a mobile robot and 2d lidar," *SoftwareX*, vol. 30, p. 102142, 2025.
- [6] A. V. Gaona and J. L. M. Flores, "Optimización de movimiento para robots autonomos mediante redes neuronales."
- [7] L. Dust, M. Ekström, R. Gu, S. Mubeen, and C. Secleanu, "A model-based methodology for automated verification of ros 2 systems," in *Proceedings of the 2024 ACM/IEEE 6th International Workshop on Robotics Software Engineering*, 2024, pp. 35–42.
- [8] J. G. Ileyay, P. M. Ababao, and G. A. Sampedro, "Enhancing machine learning performance through quantile binning for resource forecasting," in *2025 International Conference on Mobile, Military, Maritime IT Convergence (ICMIC)*. IEEE, 2025, pp. 274–277.
- [9] F. M. Corzo Acuña, "Comparación de algoritmos de aprendizaje por refuerzo: Dqn vs ppo en el juego atari amidar," 2026.
- [10] A. Bonci, F. Gaudeni, M. C. Giannini, and S. Longhi, "Robot operating system 2 (ros2)-based frameworks for increasing robot autonomy: A survey," *applied sciences*, vol. 13, no. 23, p. 12796, 2023.
- [11] N. M. AbdelAziz, G. A. Fouad, S. Al-Saeed, and A. M. Fawzy, "Deep q-network (dqn) model for disease prediction using electronic health records (ehrs)," *Sci*, vol. 7, no. 1, p. 14, 2025.